

Ampliando: Git para creadores de contenido

Guía detallada de control de versiones

JA Palazón Ferrando

2026-06-17

Contenidos

1	¿Qué es Git?	2
2	OpenCode y Git	2
3	¿Por qué Git si no programo?	2
4	Conceptos clave expandidos	3
4.1	Repositorio	3
4.2	Commit	3
4.3	Staging (área de preparación)	3
4.4	Historial	3
4.5	Rama (branch)	4
5	Flujo de trabajo detallado	4
5.1	Flujo diario	4
5.2	Deshacer cambios	4
5.3	Ramas para experimentar	5
6	Git local vs. servidores	5
7	Tabla de comandos completa	6
8	Consejos prácticos	7
9	Referencias	7

i Has visto lo básico en el paso 4

Este documento amplía lo que viste en el paso 4. Si no lo has visto aún, te recomiendo empezar por allí.

💡 ¿Para quién es esta guía?

Esta guía amplía la información sobre Git para creadores de contenido que ya tienen algo de experiencia con OpenCode. **No necesitas saber programar** para entenderla. Git es una herramienta que también sirve para gestionar documentos, no solo código.

1. ¿Qué es Git?

Git es un sistema de control de versiones. Esto significa que guarda “fotos” de tus documentos en momentos importantes. Si algo sale mal, puedes volver a cualquier foto anterior.

Analogía: piensa en Git como un álbum de fotos:

- Cada commit es una foto de tus archivos
- El historial es el álbum completo
- Puedes volver atrás en cualquier momento
- Podrías crear tus propias fotos sin conflictos, o colaborar con otros

Git no borra nada. Cada versión queda guardada para siempre (a menos que borres el repositorio entero).

2. OpenCode y Git

OpenCode puede ayudarte con muchas tareas de Git:

- Ejecutar comandos git por ti
- Explicar la salida de `git status` o `git diff`
- Sugerir reglas para `.gitignore`
- Ayudarte a redactar mensajes de commit descriptivos
- Explicar errores comunes de Git

Si no estás seguro de un comando, pregunta a OpenCode antes de ejecutarlo. Por ejemplo, puedes escribir:

“¿Qué hace este comando: `git log --oneline --graph?`”

OpenCode te explicará el resultado antes de que lo ejecutes.

3. ¿Por qué Git si no programo?

Git fue creado para programadores, pero hoy lo usan escritores, diseñadores, traductores y cualquier persona que trabaje con documentos versionados.

Ventajas para creadores de contenido:

Ventaja	Ejemplo práctico
Historial completo	Saber quién cambió qué línea y cuándo
Volver atrás	Recuperar un párrafo que borraste por error
Colaborar sin conflictos	Varios autores trabajando a la vez
Ramas de experimento	Probar una reestructuración sin afectar el original
Mensajes descriptivos	Cada cambio lleva una explicación de por qué se hizo

Si alguna vez has tenido archivos como `informe-v3-final-definitivo.docx`, `informe-v3-final-definitivo-copia.docx`, `informe-v3-final-definitivo-copia-2.docx`... Git es la solución.

4. Conceptos clave expandidos

4.1. Repositorio

Un repositorio es un directorio que Git controla. Contiene todos tus archivos y un directorio oculto `.git/` donde se guarda el historial.

```
# Crear un repositorio
mkdir mi-proyecto
cd mi-proyecto
git init
```

Después de `git init`, el directorio tiene un subdirectorio `.git/` que no debes tocar. Ahí está todo el historial.

4.2. Commit

Un commit es una “foto” de tus archivos en un momento dado. Cada commit tiene:

- Un identificador único (hash)
- Un mensaje que describe qué cambió
- El autor y la fecha
- Los archivos exactos en ese momento

```
# Crear un commit
git add documento.md
git commit -m "Añade introducción al documento"
```

El mensaje del commit debe ser claro y descriptivo. No escribas “cambio” o “update”. Mejor: “Añade sección de conclusiones” o “Corrige erratas en capítulo 2”.

4.3. Staging (área de preparación)

Antes de hacer un commit, Git tiene un área intermedia donde seleccionas qué archivos quieres incluir. Esto se llama “staging”.

```
# Añadir un archivo al staging
git add documento.md

# Añadir todos los archivos al staging
git add .

# Ver qué hay en el staging
git status
```

¿Por qué no se guardan todos los cambios directamente? Porque a veces solo quieres guardar parte de lo que has modificado. El staging te da control sobre qué entra en cada commit.

4.4. Historial

El historial es la lista ordenada de todos los commits. Puedes verlo con:

```
# Historial resumido
git log --oneline

# Historial detallado
git log
```

```
# Historial con diferencias
```

```
git log -p
```

Cada línea de `git log --oneline` muestra el hash abreviado y el mensaje del commit. Es la forma más rápida de ver qué ha pasado.

4.5. Rama (branch)

Una rama es una línea de trabajo independiente. Por defecto, tienes una rama principal (normalmente `main`). Las ramas te permiten:

- Experimentar sin tocar la versión principal
- Podrías trabajar en una funcionalidad mientras otra persona corrige errores
- Fusionar los resultados cuando estén listos

```
# Crear y cambiarse a una rama nueva
```

```
git switch -c experimento
```

```
# Volver a la rama principal
```

```
git switch main
```

```
# Fusionar la rama experimento en main
```

```
git merge experimento
```

Las ramas son como pestañas en un navegador: cada una muestra una versión diferente del mismo proyecto, y puedes cambiar entre ellas.

5. Flujo de trabajo detallado

5.1. Flujo diario

El flujo básico es siempre el mismo:

1. `git status` → ¿Qué ha cambiado?
2. `git diff` → ¿Qué exactamente?
3. `git add archivo` → Preparo el cambio
4. `git commit -m "..."` → Lo guardo en el historial

Consejo: haz commits con frecuencia. Es mejor hacer 5 commits pequeños que 1 commit gigante con 50 cambios. Los commits pequeños son más fáciles de entender y deshacer si algo sale mal.

5.2. Deshacer cambios

Si has modificado un archivo y no te gusta el resultado:

```
# Deshacer cambios en un archivo (volver al último commit)
```

```
git restore documento.md
```

```
# Guardar cambios temporalmente y limpiar el área de trabajo
```

```
git stash
```

```
# Recuperar los cambios guardados
```

```
git stash pop
```

`git restore` descarta los cambios no guardados. `git stash` los guarda temporalmente (como una nota adhesiva) y te devuelve un área de trabajo limpia.

⚠ Atención con git restore

El comando `git restore` descarta los cambios que no están en el staging. Si quieres guardar una copia de seguridad antes, usa `git stash` o simplemente copia el archivo a otro sitio.

5.3. Ramas para experimentar

Las ramas son ideales para probar cosas nuevas sin riesgo:

```
# Crear rama de experimentación
git switch -c nueva-estructura

# Hacer cambios y commits en la rama nueva
git add .
git commit -m "Prueba nueva estructura de capítulos"

# Si no te gusta, vuelves a main y borras la rama
git switch main
git branch -d nueva-estructura

# Si te gusta, la fusionas
git switch main
git merge nueva-estructura
```

Ejemplo práctico: estás escribiendo un informe y quieres probar un orden diferente de secciones. Crea una rama, mueve las secciones, y si no te convence, vuelves a la versión anterior con `git switch main`.

6. Git local vs. servidores

Git funciona en tu ordenador (local). No necesitas conexión a internet para guardar commits o ver el historial.

Sin embargo, puedes sincronizar tu repositorio con un servidor en la nube para:

- **Hacer copias de seguridad** en otro sitio
- **Colaborar** con otros (o incluso contigo mismo)
- **Acceder** desde varios ordenadores

Los servidores de Git más populares son:

Servicio	Características	Gratis
GitHub	El más popular, integrado con muchas herramientas	Sí (repositorios públicos)
GitLab	Alternativa completa, incluye CI/CD	Sí
Bitbucket	Integrado con Jira, bueno para equipos	Sí (hasta 5 usuarios)
Codeberg	Sin ánimo de lucro, alojado en la UE, sin rastreo	Sí (ilimitado)

💡 Codeberg: la opción europea

Si eres europeo o te importa la privacidad, Codeberg es una excelente alternativa:

- **Sin ánimo de lucro:** gestionado por una asociación en Berlín
- **Cumple con el RGPD:** tus datos están protegidos por la legislación europea
- **Sin rastreo:** no vende tus datos ni usa tu código para entrenar IA
- **Código abierto:** basado en Forgejo, puedes migrar fácilmente
- **Repositorios ilimitados:** públicos y privados, sin coste

i No necesitas un servidor para empezar

Puedes trabajar perfectamente sin un servidor. Solo necesitas sincronizar cuando quieras:

- Hacer copias de seguridad
- Colaborar con otros
- Trabajar desde varios ordenadores

Para sincronizar con un servidor, primero crea un repositorio en GitHub (o similar) y luego:

```
# Añadir el servidor como origen
git remote add origin https://github.com/tu-usuario/tu-repositorio.git

# Subir tus commits
git push -u origin main

# Bajar cambios del servidor
git pull
```

7. Tabla de comandos completa

Comando	Para qué sirve
<code>git init</code>	Crear un repositorio nuevo
<code>git config --global user.name "Nombre"</code>	Configurar tu nombre
<code>git config --global user.email "email"</code>	Configurar tu correo
<code>git status</code>	Ver el estado actual del repositorio
<code>git diff</code>	Ver cambios sin committed
<code>git add archivo</code>	Añadir un archivo al staging
<code>git add .</code>	Añadir todos los archivos al staging
<code>git commit -m "mensaje"</code>	Guardar un commit con mensaje
<code>git log --oneline</code>	Historial resumido
<code>git log</code>	Historial detallado
<code>git show HASH</code>	Ver qué cambió en un commit concreto
<code>git restore archivo</code>	Deshacer cambios en un archivo
<code>git stash</code>	Guardar cambios temporalmente
<code>git stash pop</code>	Recuperar cambios guardados
<code>git switch -c NOMBRE</code>	Crear y cambiarse a una rama nueva

Comando	Para qué sirve
<code>git switch NOMBRE</code>	Cambiar a una rama existente
<code>git merge NOMBRE</code>	Fusionar una rama en la actual
<code>git branch</code>	Ver todas las ramas
<code>git branch -d NOMBRE</code>	Borrar una rama
<code>git remote add ORIGIN URL</code>	Añadir un servidor remoto
<code>git push</code>	Subir commits al servidor
<code>git pull</code>	Bajar cambios del servidor

8. Consejos prácticos

1. **Haz commits pequeños y frecuentes.** Es mejor guardar 5 cambios pequeños que 1 cambio gigante. Los commits pequeños son más fáciles de entender y deshacer.
2. **Escribe mensajes descriptivos.** No uses “cambio” o “update”. Describe qué has hecho: “Añade sección de conclusiones”, “Corrige erratas en capítulo 2”, “Reestructura apartado 3”.
3. **Usa .gitignore desde el principio.** Crea un archivo `.gitignore` para que Git ignore archivos temporales, HTML generados y otros archivos que no necesitas versionar.
4. **No tengas miedo de experimentar.** Con las ramas, puedes probar cosas nuevas sin riesgo. Si no te gusta, vuelves atrás.
5. **Sincroniza con frecuencia.** Si trabajas con un servidor, sube tus cambios regularmente para no perder trabajo.
6. **Revisa antes de commit.** Usa `git status` y `git diff` antes de cada commit para asegurarte de que estás guardando lo que quieres.

Consejo

En resumen: Git es tu aliado para no perder trabajo. Piensa en cada commit como una foto de tus documentos. Si algo sale mal, siempre puedes volver a la foto anterior. Y si quieres experimentar, crea una rama nueva y juega sin miedo.

9. Referencias

- [Pro Git](#) — Libro oficial de Git en español (gratuito)
- [GitHub Cheat Sheet](#) — Resumen de comandos útiles
- [Learn Git Branching](#) — Tutorial interactivo para practicar ramas